```
#!pip install spacy
#(we have 3 type of model is available called small, medium & large)
```

```
#!python -m spacy download en_core_web_sm
```

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("data science and ai has greate career ahead")
```

```
#import PyQt5
```

```
doc
```

```
data science and ai has greate career ahead
```

```
for token in doc:
    print(token.text)
```

```
data
science
and
ai
has
greate
career
ahead
```

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple is looking at buying U.K. startup for $1 billion")

for token in doc:
    print(token.text, token.lemma_, token.pos_, token.tag_, token.dep_,
            token.shape_, token.is_alpha, token.is_stop)
```

```
Apple Apple PROPN NNP nsubj Xxxxx True False
is be AUX VBZ aux xx True True
looking look VERB VBG ROOT xxxx True False
at at ADP IN prep xx True True
buying buy VERB VBG pcomp xxxx True False
U.K. U.K. PROPN NNP nsubj X.X. False False
startup startup VERB VBD ccomp xxxx True False
for for ADP IN prep xxx True True
$ $ SYM $ quantmod $ False False
1 1 NUM CD compound d False False
billion billion NUM CD pobj xxxx True False
```

```
for token in doc:
    print(token.pos_)
```

```
PROPN
AUX
VERB
ADP
VERB
PROPN
VERB
ADP
SYM
NUM
NUM
```

```
for token in doc:
    print(token.text, token.pos_)
```

```
Apple PROPN
is AUX
looking VERB
at ADP
buying VERB
U.K. PROPN
startup VERB
for ADP
```

```
$ SYM
1 NUM
billion NUM
```

```
for token in doc:
    print(token.text, token.pos_, token.lemma_)
```

```
Apple PROPN Apple
is AUX be
looking VERB look
at ADP at
buying VERB buy
U.K. PROPN U.K.
startup VERB startup
for ADP for
$ SYM $
1 NUM 1
billion NUM billion
```

```
text = """There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on.
An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a giv
Image collection summarization is another application example of automatic summarization. It consists in selecting a representa
```

```
text
```

```
'There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. The firs
t is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or s
ets of images, or videos, news stories etc.). The second is query relevant summarization, sometimes called query-based summariz
ation, which summarizes objects specific to a query. Summarization systems are able to create both query relevant text summarie
s and generic machine-generated summaries depending on what the user needs.\nAn example of a summarization problem is document
summarization, which attempts to automatically produce an abstract from a given document. Sometimes one might be interested in
generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of a
```

```
import spacy
from spacy.lang.en.stop_words import STOP_WORDS
from string import punctuation
```

```
stopwords = list(STOP_WORDS)
stopwords
```

```
label
```

```
 'she',
 'hereafter',
 'none',
 '’s',
 'doing',
 'off',
 'unless',
 'did',
 'much',
 'yourselves',
 'using',
 'whither',
 'really',
 'meanwhile',
 'how',
 'somewhere',
 'may',
 '‘s',
 'former']
```

```
len(stopwords)
```

```
326
```

```
nlp = spacy.load('en_core_web_sm')
```

```
text
```

```
'There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. The firs
t is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or s
ets of images, or videos, news stories etc.). The second is query relevant summarization, sometimes called query-based summariz
ation, which summarizes objects specific to a query. Summarization systems are able to create both query relevant text summarie
s and generic machine-generated summaries depending on what the user needs.\nAn example of a summarization problem is document
summarization, which attempts to automatically produce an abstract from a given document. Sometimes one might be interested in
generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of a
```

```
doc = nlp(text)
doc
```

```
There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. The first
is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or
sets of images, or videos, news stories etc.). The second is query relevant summarization, sometimes called query-based
summarization, which summarizes objects specific to a query. Summarization systems are able to create both query relevant text
summaries and generic machine-generated summaries depending on what the user needs.
An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
given document. Sometimes one might be interested in generating a summary from a single source document, while others can use
multiple source documents (for example, a cluster of articles on the same topic). This problem is called multi-document
summarization. A related application is summarizing news articles. Imagine a system, which automatically pulls together news
articles on a given topic (from the web), and concisely represents the latest news as a summary.
Image collection summarization is another application example of automatic summarization. It consists in selecting a
representative set of images from a larger set of images.[4] A summary in this context is useful to show the most
representative images of results in an image collection exploration system. Video summarization is a related domain, where the
system automatically creates a trailer of a long video. This also has applications in consumer or personal videos, where one
might want to skip the boring or repetitive actions. Similarly, in surveillance videos, one would want to extract important and
suspicious activity, while ignoring all the boring and redundant frames captured
```

```
# lets get the tokens from text
tokens = [token.text for token in doc]
print(tokens)
#when we execute everythihg we created tokens from the text & not removed any of the stopwords & didnt cleaned the data
```

```
['There', 'are', 'broadly', 'two', 'types', 'of', 'extractive', 'summarization', 'tasks', 'depending', 'on', 'what', 'the', 'sum
```

```
# TEAM PLEASE INSTALL FR_CORE_WEB_SM (PLEASE DONWLOAD THE STOPWONR , LEN OF THE STOPWORDS)
```

```
tokens
```

```
'This'
```

```
 'this',
 'also',
 'has',
 'applications',
 'in',
 'consumer',
 'or',
 'personal',
 'videos',
 ',',
 'where',
 'one',
 'might',
 'want',
 'to',
 'skip',
 'the',
 'boring',
 'or',
 'repetitive',
 'actions',
 '.',
 'Similarly',
 ',',
 'in',
 'surveillance',
 'videos',
 ',',
 'one',
 'would',
 'want',
 'to',
 'extract',
 'important',
 'and',
 'suspicious',
 'activity',
 ',',
 'while',
 'ignoring',
 'all',
 'the',
 'boring',
 'and',
 'redundant',
 'frames',
 'captured']
```

```
len(tokens)
```

```
322
```

```
punctuation # also called as noisy characters
```

```
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

```
doc
```

```
There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on. The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or sets of images, or videos, news stories etc.). The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a query. Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending on what the user needs.
An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document. Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of articles on the same topic). This problem is called multi-document summarization. A related application is summarizing news articles. Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents the latest news as a summary.
Image collection summarization is another application example of automatic summarization. It consists in selecting a representative set of images from a larger set of images.[4] A summary in this context is useful to show the most representative images of results in an image collection exploration system. Video summarization is a related domain, where the system automatically creates a trailer of a long video. This also has applications in consumer or personal videos, where one might want to skip the boring or repetitive actions. Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring and redundant frames captured
```

```
#we have to calcualte the freaquency of each and every word, how many time word is repetation in text

word_frequencies = {}

for word in doc:
    if word.text.lower() not in stopwords:
```

```
        if word.text.lower() not in punctuation:
            if word.text not in word_frequencies.keys():
                word_frequencies[word.text] = 1
            else:
                word_frequencies[word.text] += 1
```

```
word_frequencies
#print(word_frequencies)
```

```
 'given': 2,
 'interested': 1,
 'generating': 1,
 'single': 1,
 'source': 2,
 'use': 1,
 'multiple': 1,
 'cluster': 1,
 'articles': 3,
 'topic': 2,
 'multi': 1,
 'related': 2,
 'application': 2,
 'summarizing': 1,
 'Imagine': 1,
 'system': 3,
 'pulls': 1,
 'web': 1,
 'concisely': 1,
 'represents': 1,
 'latest': 1,
 'Image': 1,
 'automatic': 1,
 'consists': 1,
 'selecting': 1,
 'representative': 2,
 'set': 2,
 'larger': 1,
 'images.[4': 1,
 'context': 1,
 'useful': 1,
 'results': 1,
 'image': 1,
 'exploration': 1,
 'Video': 1,
 'domain': 1,
 'creates': 1,
 'trailer': 1,
 'long': 1,
 'video': 1,
 'applications': 1,
 'consumer': 1,
 'personal': 1,
 'want': 2,
 'skip': 1,
 'boring': 2,
 'repetitive': 1,
 'actions': 1,
 'Similarly': 1,
 'surveillance': 1,
 'extract': 1,
 'important': 1,
 'suspicious': 1,
 'activity': 1,
 'ignoring': 1,
 'redundant': 1,
 'frames': 1,
 'captured': 1}
```

```
len(word_frequencies)
```

```
103
```

```
word_frequencies
```

```

 'clustering': 1
```

```
 'cluster': 1,
 'articles': 3,
 'topic': 2,
 'multi': 1,
 'related': 2,
 'application': 2,
 'summarizing': 1,
 'Imagine': 1,
 'system': 3,
 'pulls': 1,
 'web': 1,
 'concisely': 1,
 'represents': 1,
 'latest': 1,
 'Image': 1,
 'automatic': 1,
 'consists': 1,
 'selecting': 1,
 'representative': 2,
 'set': 2,
 'larger': 1,
 'images.[4': 1,
 'context': 1,
 'useful': 1,
 'results': 1,
 'image': 1,
 'exploration': 1,
 'Video': 1,
 'domain': 1,
 'creates': 1,
 'trailer': 1,
 'long': 1,
 'video': 1,
 'applications': 1,
 'consumer': 1,
 'personal': 1,
 'want': 2,
 'skip': 1,
 'boring': 2,
 'repetitive': 1,
 'actions': 1,
 'Similarly': 1,
 'surveillance': 1,
 'extract': 1,
 'important': 1,
 'suspicious': 1,
 'activity': 1,
 'ignoring': 1,
 'redundant': 1,
 'frames': 1,
 'captured': 1}
```

```
max_frequency = max(word_frequencies.values())
max_frequency
```

```
11
```

```
#to get normalized/weighted frequencies you should devide all frequencies with 11
for word in word_frequencies.keys():
    word_frequencies[word] = word_frequencies[word]/max_frequency
```

```
#print(word_frequencies)
word_frequencies
#this is the normalized frequencies of each word
```

```
 'capsicum': 0.09090909090909091
```

```
 'concisely': 0.09090909090909091,
 'represents': 0.09090909090909091,
 'latest': 0.09090909090909091,
 'Image': 0.09090909090909091,
 'automatic': 0.09090909090909091,
 'consists': 0.09090909090909091,
 'selecting': 0.09090909090909091,
 'representative': 0.18181818181818182,
 'set': 0.18181818181818182,
 'larger': 0.09090909090909091,
 'images.[4': 0.09090909090909091,
 'context': 0.09090909090909091,
 'useful': 0.09090909090909091,
 'results': 0.09090909090909091,
 'image': 0.09090909090909091,
 'exploration': 0.09090909090909091,
 'Video': 0.09090909090909091,
 'domain': 0.09090909090909091,
 'creates': 0.09090909090909091,
 'trailer': 0.09090909090909091,
 'long': 0.09090909090909091,
 'video': 0.09090909090909091,
 'applications': 0.09090909090909091,
 'consumer': 0.09090909090909091,
 'personal': 0.09090909090909091,
 'want': 0.18181818181818182,
 'skip': 0.09090909090909091,
 'boring': 0.18181818181818182,
 'repetitive': 0.09090909090909091,
 'actions': 0.09090909090909091,
 'Similarly': 0.09090909090909091,
 'surveillance': 0.09090909090909091,
 'extract': 0.09090909090909091,
 'important': 0.09090909090909091,
 'suspicious': 0.09090909090909091,
 'activity': 0.09090909090909091,
 'ignoring': 0.09090909090909091,
 'redundant': 0.09090909090909091,
 'frames': 0.09090909090909091,
 'captured': 0.09090909090909091}
```

```
sentence_tokens = [sent for sent in doc.sents]
sentence_tokens
```

```
[There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on.,
 The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether
documents, or sets of images, or videos, news stories etc.).,
 The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a
query.,
 Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending
on what the user needs.,
 An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
given document.,
 Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source
documents (for example, a cluster of articles on the same topic).,
 This problem is called multi-document summarization.,
 A related application is summarizing news articles.,
 Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents
the latest news as a summary.,
 Image collection summarization is another application example of automatic summarization.,
 It consists in selecting a representative set of images from a larger set of images.[4],
 A summary in this context is useful to show the most representative images of results in an image collection exploration
system.,
 Video summarization is a related domain, where the system automatically creates a trailer of a long video.,
 This also has applications in consumer or personal videos, where one might want to skip the boring or repetitive actions.,
 Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring
and redundant frames captured]
```

```
len(sentence_tokens)
```

```
15
```

```
sentence_tokens
```

```
[There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on.,
 The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether
documents, or sets of images, or videos, news stories etc.).,
 The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a
query.,
 Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending
on what the user needs.,
 An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
```

```
given document.,
 Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of articles on the same topic).,
 This problem is called multi-document summarization.,
 A related application is summarizing news articles.,
 Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents the latest news as a summary.,
 Image collection summarization is another application example of automatic summarization.,
 It consists in selecting a representative set of images from a larger set of images.[4],
 A summary in this context is useful to show the most representative images of results in an image collection exploration system.,
 Video summarization is a related domain, where the system automatically creates a trailer of a long video.,
 This also has applications in consumer or personal videos, where one might want to skip the boring or repetitive actions.,
 Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring and redundant frames captured]
```

```
# we are going to calculate the sentence score, to calculate the sentence score
sentence_scores = {}

for sent in sentence_tokens:
    for word in sent:
        if word.text.lower() in word_frequencies.keys():
            if sent not in sentence_scores.keys():
                sentence_scores[sent] = word_frequencies[word.text.lower()]
            else:
                sentence_scores[sent] += word_frequencies[word.text.lower()]
```

```
sentence_scores
```

```
{There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on.: 2.818181818181818,
 The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or sets of images, or videos, news stories etc.).: 3.9999999999999987,
 The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a query.: 3.909090909090909,
 Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending on what the user needs.: 3.2727272727272716,
 An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document.: 3.9999999999999996,
 Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source documents (for example, a cluster of articles on the same topic).: 2.545454545454545,
 This problem is called multi-document summarization.: 1.8181818181818183,
 A related application is summarizing news articles.: 1.0909090909090908,
 Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents the latest news as a summary.: 2.9090909090909087,
 Image collection summarization is another application example of automatic summarization.: 2.909090909090909,
 It consists in selecting a representative set of images from a larger set of images.[4]: 1.1818181818181817,
 A summary in this context is useful to show the most representative images of results in an image collection exploration system.: 1.818181818181818,
 Video summarization is a related domain, where the system automatically creates a trailer of a long video.: 2.2727272727272725,
 This also has applications in consumer or personal videos, where one might want to skip the boring or repetitive actions.: 1.1818181818181817,
 Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring and redundant frames captured: 1.4545454545454544}
```

```
#lets say our case study was 30% sentence with maximum scores
from heapq import nlargest
```

```
select_length = int(len(sentence_tokens)*0.4)
select_length
```

```
6
```

```
#we have to select maximum 4 sentences out of all sentences
summary = nlargest(select_length,sentence_scores, key = sentence_scores.get)
```

```
summary
```

```
[An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a given document.,
 The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether documents, or sets of images, or videos, news stories etc.).,
 The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a query.,
 Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending on what the user needs.,
```

```
 Image collection summarization is another application example of automatic summarization.,
 Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents
the latest news as a summary.]
```

```
sentence_scores
```

```
{There are broadly two types of extractive summarization tasks depending on what the summarization program focuses on.:
2.818181818181818,
 The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether
documents, or sets of images, or videos, news stories etc.).: 3.9999999999999987,
 The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to a
query.: 3.909090909090909,
 Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending
on what the user needs.: 3.2727272727272716,
 An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
given document.: 3.9999999999999996,
 Sometimes one might be interested in generating a summary from a single source document, while others can use multiple source
documents (for example, a cluster of articles on the same topic).: 2.545454545454545,
 This problem is called multi-document summarization.: 1.8181818181818183,
 A related application is summarizing news articles.: 1.0909090909090908,
 Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents
the latest news as a summary.: 2.9090909090909087,
 Image collection summarization is another application example of automatic summarization.: 2.909090909090909,
 It consists in selecting a representative set of images from a larger set of images.[4]: 1.1818181818181817,
 A summary in this context is useful to show the most representative images of results in an image collection exploration
system.: 1.818181818181818,
 Video summarization is a related domain, where the system automatically creates a trailer of a long video.:
2.2727272727272725,
 This also has applications in consumer or personal videos, where one might want to skip the boring or repetitive actions.:
1.1818181818181817,
 Similarly, in surveillance videos, one would want to extract important and suspicious activity, while ignoring all the boring
and redundant frames captured: 1.4545454545454544}
```

```
# if i need to combine these top 3 sentencs then

final_summary = [word.text for word in summary]
```

```
final_summary
```

```
['An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a
given document.',
 'The first is generic summarization, which focuses on obtaining a generic summary or abstract of the collection (whether
documents, or sets of images, or videos, news stories etc.).',
 'The second is query relevant summarization, sometimes called query-based summarization, which summarizes objects specific to
a query.',
 'Summarization systems are able to create both query relevant text summaries and generic machine-generated summaries depending
on what the user needs.\n',
 'Image collection summarization is another application example of automatic summarization.',
 'Imagine a system, which automatically pulls together news articles on a given topic (from the web), and concisely represents
the latest news as a summary.\n']
```

```
print(summary) # we get the final summary by our model
```

```
[An example of a summarization problem is document summarization, which attempts to automatically produce an abstract from a giv
, Image collection summarization is another application example of automatic summarization., Imagine a system, which automatical
]
```

```
# client assigne this work nlp enginner -- share this text to cleint for evaluiton
# reduce manual work
# still pen adn paper
```

```
# how to extra text from paper
# chate generati ai whoud
# RAW PAER PDF
#-- PDF to text ( nlp techniqeu)
# image (ocr)
```

```
Start coding or generate with AI.
```